

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
SEGUNDO SEMESTRE 2026



MANUAL TÉCNICO – APLICACIÓN WEB CELIX

Ixcolín Orozco, Javier Eduardo

Carné: 202132163

DOCENTE:

Ing. Mario Ramírez Tobar

VERSIÓN 1.0

Contenido

Tecnología Utilizadas	1
PHP (8.2.10):	1
Laravel (12):	1
Git (2.55.0.windows.3):	1
GitHub:	1
Beekeeper (6.1.2):	1
Bootstrap:	1
Antigravity (2.15.0):	1
Draw.io:	2
PostgreSQL (18):	2
Docker Desktop (4.91.0):	2
Primeros Pasos	3
Requisitos	3
Instrucciones	3
Paso 1. Ubicarse en la raíz del proyecto	3
Paso 2. Construir e iniciar los contenedores	3
Paso 3. Instalar las dependencias de PHP	3
Paso 4. Verificar el archive del entorno y clave de aplicación	3
Paso 5. Ejecutar migraciones y datos iniciales (Seeders)	3
Paso 6. Compilar recursos frontend (opcional / si se requieren assets)	3
Credenciales y Accesos Iniciales	4
Comandos de Utilidad y Mantenimiento	4
ENDPOINTS, Request y Responses	5
1. MÓDULO PÚBLICO Y AUTENTICACIÓN	5
1.1. Mostrar Formulario de Inicio de Sesión	5
- Método y URL: GET / GET /login	5
1.2. Iniciar Sesión (Login)	5
1.3. Cerrar Sesión (Logout)	5
1.4. Vista de Recuperación de Contraseña	5
1.5. Seguimiento Público de Reparación (Escaneo QR)	6
1.6. Descarga Pública de Comprobante PDF (vía QR)	6
2. PERFIL UNIVERSAL DE USUARIO (Común a todos los roles)	6

2.1.	Ver Perfil del Usuario Autenticado	6
2.2.	Actualizar Contraseña Propia.....	6
3.	MÓDULO DE ADMINISTRACIÓN (role:ADMINISTRADOR, Prefijo /admin)	7
3.1.	Dashboard de Administración.....	7
3.2.	Listado y Gestión de Usuarios (HTML).....	7
3.3.	Listar Usuarios en Formato JSON (Para AJAX o integración)	7
3.4.	Crear Nuevo Usuario	8
3.5.	Actualizar Datos de Usuario	9
3.6.	Cambiar Contraseña de Usuario por Administrador	9
3.7.	Activar / Desactivar Usuario (Toggle State).....	10
3.8.	Configuración del Sistema (Tipos de Dispositivos)	10
3.9.	Registrar Tipo de Dispositivo	10
3.10.	Actualizar Tipo de Dispositivo.....	11
3.11.	Activar / Desactivar Tipo de Dispositivo	11
4.	MÓDULO DE RECEPCIÓN E INVENTARIO (role:ADMINISTRADOR,RECEPCIONISTA, Prefijo /receptionist)	11
4.1.	Panel de Inicio de Recepción (Ventas y Nuevo Ticket)	11
4.2.	Registrar Venta de Productos (POS y Kardex)	12
4.3.	Recepción de Dispositivo (Crear Ticket de Taller y QR)	12
4.4.	Descarga de Ticket en PDF (Recepción).....	14
4.5.	Módulo de Inventario (Vista de Productos y Categorías)	14
4.6.	Registrar Producto en Inventario	14
4.7.	Actualizar Producto en Inventario.....	15
4.8.	Activar / Desactivar Producto	15
4.9.	Registrar Categoría de Productos	15
4.10.	Actualizar Categoría de Productos	16
4.11.	Activar / Desactivar Categoría	16
4.12.	Registrar Movimiento de Inventario Manual (Kardex)	16
4.13.	Historial de Movimientos de Inventario (Kardex).....	16
4.14.	Módulo de Entregas (Tickets Listos o Finalizados).....	16
4.15.	Procesar Entrega y Liquidación de Dispositivo	17
5.	MÓDULO TÉCNICO (role:ADMINISTRADOR,TECNICO, PREFIJO /technician)	17
5.1.	Panel de Inicio del Técnico.....	17

5.2.	Avanzar Estado del Ticket.....	17
5.3.	Registrar Nota Técnica en Bitácora del Ticket	18
5.4.	Listar Notas de un Ticket (Historial de Bitácora)	19
Diagramas		20
Diagrama de Clases		20
Diagrama de Arquitectura e Infraestructura.....		21
Modelo Entidad-Relación (E/R)		22
Estructura de Paqueterías		23
Modelos de Procesos Técnicos.....		24
Historias de Usuarios.....		26
Épica 1: Autenticación, Seguridad y Gestión de Usuarios		26
SCRUM-01: Inicio de Sesión Seguro con RBAC y Username		26
SCRUM-02: Creación y Gestión Administrativa de Usuarios		26
SCRUM-03: Recuperación de Contraseña y Autenticación en Dos Factores (2FA)		26
Épica 2: Gestión del Flujo de Tickets de Reparación.....		27
SCRUM-04: Recepción de Dispositivos y Creación de Tickets con Multidispositivo		27
SCRUM-05: Bandeja Técnica, Diagnóstico y Actualización de Bitácora		27
SCRUM-06: Consulta Pública y Seguimiento de Reparación mediante Código QR.....		28
SCRUM-07: Entrega de Equipo, Cobro de Mano de Obra y Cierre de Ticket.....		28
Épica 3: Control y Gestión de Inventario		28
SCRUM-08: Catálogo de Accesorios y Semáforo Visual de Existencias		29
SCRUM-09: Control de Entradas/Salidas y Alerta de Stock Mínimo		29
Épica 4: Control Financiero Básico y Ventas		29
SCRUM-10: Venta de Accesorios Maestro-Detalle de Mostrador		29
SCRUM-11: Registro y Clasificación de Egresos Operativos.....		30
Épica 5: Reportería, Métricas y Dashboard Ejecutivo		30
SCRUM-12: Dashboard Gerencial con Métricas Operativas y Financieras		30
SCRUM-13: Módulo de Reportes con Exportación Obligatoria a PDF y Excel.....		31
Casos de Uso		32
CU-01: Autenticación e Inicio de Sesión.....		32
Flujo Principal de Eventos		32
Flujos Alternativos y Excepciones		32
CU-02: Gestión de Cuentas de Usuario		33

Flujo Principal de Eventos	33
Flujos Alternativos (Desactivación de Usuario).....	33
CU-03: Admisión de Dispositivo y Emisión de Ticket con Código QR.....	34
Flujo Principal de Eventos	34
Flujos Alternativos	35
CU-04: Diagnóstico y Actualización de Estado Técnico en Taller	35
Flujo Principal de Eventos	35
Flujos Alternativos y Excepciones	36
CU-05: Entrega y Liquidación de Dispositivo Reparado	36
Flujo Principal de Eventos	37
CU-06: Venta de Mostrador (POS) y Control Automático de Kardex	37
Flujos Alternativos	38
CU-07: Consulta Pública de Estado Mediante Código QR	38
Flujos Alternativos	39
CU-08: Consulta de Dashboard y Exportación de Reportes	39

Tecnología Utilizadas

PHP (8.2.10):

Es un lenguaje de programación de código abierto que se ejecuta en el lado del servidor para crear sitios web dinámicos.

Laravel (12):

Es un framework de PHP de código abierto que sirve para desarrollar aplicaciones y servicios web de manera rápida, ordenada y elegante.

Un framework (o entorno de trabajo) es un conjunto de herramientas y librerías preestablecidas que evita tener que escribir código desde cero para tareas comunes.

Git (2.55.0.windows.3):

Es un sistema de control de versiones distribuido y de código abierto que registra los cambios realizados en el código fuente de un proyecto a lo largo del tiempo.

GitHub:

Es una plataforma e la nube que permite a los desarrolladores almacenar, administrar y colaborar en sus proyectos de código fuente utilizando el sistema de control de versiones de Git.

Beekeeper (6.1.2):

Es un editor SQL moderno, limpio y de código abierto para gestionar bases de datos. Funciona en Windows, macOS y Linux.

Bootstrap:

Es un popular framework o kit de herramientas de código abierto para el desarrollo web front-end (la parte visual de un sitio) que permite crear páginas responsivas y adaptables a cualquier pantalla de forma rápida.

Antigravity (2.15.0):

Es una plataforma de desarrollo de software impulsada por inteligencia artificial y diseñada para la era centrada en agentes autónomos.

Inicialmente presentado como un entorno de desarrollo integrado (IDE) derivado de Visual Studio Code con capacidades de Gemini se ha expandido a un ecosistema completo (incluyendo aplicación de escritorio independiente y CLI) que funciona como centro de mando para gestionar múltiples agentes de IA en paralelo.

Draw.io:

Es una herramienta en línea gratuita y de código abierto para crear diagramas, esquemas y mapas conceptuales.

PostgreSQL (18):

Es un sistema de gestión de bases de datos relacional y orientada a objetos, de código abierto y gratuito.

Docker Desktop (4.91.0):

Es una aplicación fácil de usar que permite crear, ejecutar y administrar aplicaciones en contenedores directamente desde tu computadora con Windows, Mac o Linux.

Un contenedor es un paquete ligero que incluye todo lo que una aplicación necesita para funcionar (código, herramientas y configuración), aislándola del resto del sistema.

Primeros Pasos

Requisitos

Contar con las siguientes tecnologías instaladas en el equipo con el que se trabajará:

- Docker Desktop
- Git
- Navegador web de su preferencia

Instrucciones

Paso 1. Ubicarse en la raíz del proyecto

Abre una terminal (PowerShell, Bash o CMD) en la raíz del repositorio donde reside el archivo `docker-compose.yml`.

Paso 2. Construir e iniciar los contenedores

El siguiente comando descargará la imagen de PostgreSQL 18, compilará el contenedor de PHP 8.3 CLI con sus extensiones e iniciará los servicios en segundo plano:

```
docker compose up -d --build
```

Paso 3. Instalar las dependencias de PHP

Dentro del contenedor de la aplicación, ejecutar el siguiente comando:

```
docker compose exec app composer install
```

Paso 4. Verificar el archivo del entorno y clave de aplicación

El script de entrada del contenedor crea automáticamente el archivo `.env` basado en `.env.docker.example` si no existe. Para asegurar que la clave única esté generada, ejecuta:

```
docker compose exec app php artisan key:generate
```

Paso 5. Ejecutar migraciones y datos iniciales (Seeders)

Crea la estructura de tablas e inserta los roles, usuario administrador inicial y tipos de dispositivos ejecutando el siguiente comando:

```
docker compose exec app php artisan migrate --seed
```

Paso 6. Compilar recursos frontend (opcional / si se requieren assets)

```
docker compose exec app npm install
```

```
docker compose exec app npm run build
```

Credenciales y Accesos Iniciales

Por motivos de seguridad, se reserva el acceso a las credenciales de acceso para el sistema y para la base de datos. En caso de querer solicitar información sobre este apartado, contactarse con la persona propietaria del repositorio.

Comandos de Utilidad y Mantenimiento

Ver estado de los servicios:

- `docker compose ps`

Ver logs del sistema:

- `docker compose logs -f app`
- `docker compose logs -f db`

Detener los contenedores:

- `docker compose down`

Reiniciar limpiando volúmenes de base de datos (Restablecimiento total):

- `docker compose down -v`
- `docker compose up -d`
- `docker compose exec app php artisan migrate --seed`

ENDPOINTS, Request y Responses

1. MÓDULO PÚBLICO Y AUTENTICACIÓN

1.1. Mostrar Formulario de Inicio de Sesión

- Método y URL: GET / | GET /login
- Acceso: Público (Middleware guest)
- Request: Ninguno
- Response (200 OK): Renderiza la vista Blade login.blade.php

1.2. Iniciar Sesión (Login)

- Método y URL: POST /login
- Acceso: Público (Middleware guest)
- Request (Form Data / URL Encoded):

```
{  
  "username": "string (Requerido)",  
  "password": "string (Requerido)",  
  "remember": "boolean (Opcional, default: false)"  
}
```

- Response:
- 302 Found (Redirección exitosa): Redirige a la ruta prevista o al Dashboard según el rol del usuario:
- ADMINISTRADOR -> /admin/home
- RECEPCIONISTA -> /receptionist/reception
- TECNICO -> /technician/technician
- 302 Found (Error de validación / credenciales erróneas): Redirección de regreso a /login con errores en sesión (errors) y flash message 'error'.

1.3. Cerrar Sesión (Logout)

- Método y URL: POST /logout
- Acceso: Autenticado (Middleware auth)
- Request: Token CSRF en header o body.
- Response (302 Found): Invalida la sesión actual, regenera el token CSRF y redirige a la ruta login.

1.4. Vista de Recuperación de Contraseña

- Método y URL: GET /forgot-password

- Acceso: Público (Middleware guest)
- Request: Ninguno.
- Response (200 OK): Renderiza la vista Blade public.recover-password.

1.5. Seguimiento Público de Reparación (Escaneo QR)

- Método y URL: GET /seguimiento/{token}
- Acceso: Público (sin autenticación)
- Request:
- Parámetro de ruta: token (string UUID del ticket).
- Response:
- 200 OK: Renderiza la vista Blade public.ticket-tracking con el avance del stepper de etapas (Recibido, Diagnóstico, Reparación, Finalizado, Entregado), datos del cliente, dispositivo y notas públicas.
- 404 Not Found: Vista public.not-found si el UUID es inválido o no existe en la base de datos.

1.6. Descarga Pública de Comprobante PDF (vía QR)

- Método y URL: GET /seguimiento/{token}/pdf
- Acceso: Público (sin autenticación)
- Request:
- Parámetro de ruta: token (string UUID del ticket).
- Response:
- 200 OK (Application/pdf): Descarga del archivo ticket-celix-TK-{id}.pdf con la etiqueta técnica y talón para cliente.
- 404 Not Found: Si el token no es un UUID válido o no se encuentra el ticket.

2. PERFIL UNIVERSAL DE USUARIO (Común a todos los roles)

2.1. Ver Perfil del Usuario Autenticado

- Método y URL: GET /perfil
- Acceso: Autenticado (auth)
- Request: Ninguno.
- Response (200 OK): Renderiza la vista profile.show adaptando el layout al rol del usuario.

2.2. Actualizar Contraseña Propia

- Método y URL: PUT /perfil/password
- Acceso: Autenticado (auth)
- Request (Form Data o JSON):

```

{
  "current_password": "string (Requerido, debe coincidir con la contraseña actual)",
  "new_password": "string (Requerido, min:8, diferente a la actual)",
  "new_password_confirmation": "string (Requerido, igual a new_password)"
}

- Response:
- 200 OK (AJAX / JSON):

{
  "success": true,
  "message": "¡Contraseña modificada exitosamente!"
}

- 302 Found (Web tradicional): Redirige a /perfil con flash message success.
- 422 Unprocessable Entity: Errores de validación si la clave actual no coincide o no cumple requisitos.

```

3. MÓDULO DE ADMINISTRACIÓN (role:ADMINISTRADOR, Prefijo /admin)

3.1. Dashboard de Administración

- Método y URL: GET /admin | GET /admin/home
- Response (200 OK): Renderiza admin.dashboard.

3.2. Listado y Gestión de Usuarios (HTML)

- Método y URL: GET /admin/users
- Request (Query Params opcionales):
 - search: término de búsqueda (nombre, apellido, correo, username).
 - role: ID del rol para filtrar.
 - state: 'active' | 'inactive' | ''.
 - sort_by: campo de ordenamiento.
 - sort_direction: 'asc' | 'desc'.
 - per_page: cantidad por página (default: 5).
- Response (200 OK): Renderiza la vista admin.users.index con usuarios paginados y métricas de cuentas activas/inactivas.

3.3. Listar Usuarios en Formato JSON (Para AJAX o integración)

- Método y URL: GET /admin/users/list
- Request (Query Params): Mismos filtros que en la sección 3.2.
- Response (200 OK - JSON):

```
{
  "current_page": 1,
  "data": [
    {
      "id": 1,
      "id_rol": 1,
      "name": "Nelson",
      "lastname": "Ixcolín",
      "username": "administrador",
      "email": "iservicesanmarcos@gmail.com",
      "state": true,
      "role": { "id_rol": 1, "name": "ADMINISTRADOR" }
    }
  ],
  "total": 1
}
```

3.4. Crear Nuevo Usuario

- Método y URL: POST /admin/users/create
- Request (Form Data o JSON):

```
{
  "name": "string (Requerido, max:100)",
  "lastname": "string (Requerido, max:100)",
  "username": "string (Requerido, max:50, único)",
  "email": "string (Requerido, email, max:150, único)",
  "id_rol": "integer (Requerido, existe en roles: 1=ADMINISTRADOR, 2=RECEPCIONISTA, 3=TECNICO)",
}
```

```

"password": "string (Requerido, min:8)",
"password_confirmation": "string (Requerido)",
"state": "boolean (Opcional, default: true)"
}

- Response:
- 201 Created (JSON / AJAX):

{
  "success": true,
  "message": "Usuario registrado exitosamente.",
  "user": { "id": 2, "name": "Juan", "username": "jperedas", "...": "..." }
}

- 302 Found: Redirige a /admin/users con mensaje flash success.

```

3.5. Actualizar Datos de Usuario

```

- Método y URL: PUT /admin/users/{id}
- Request:
- Parámetro de ruta: id (entero, ID de usuario).
- Body:

{
  "name": "string (Requerido, max:100)",
  "lastname": "string (Requerido, max:100)",
  "id_rol": "integer (Requerido, existe en roles)",
  "state": "boolean (Opcional)"
}

- Response:
- 200 OK (JSON): {"success": true, "message": "Información de usuario actualizada exitosamente.", "user": {...}}
- 302 Found: Redirección a /admin/users.

```

3.6. Cambiar Contraseña de Usuario por Administrador

```

- Método y URL: PATCH /admin/users/{id}/password

```

- Request:

```
{
  "password": "string (Requerido, min:8)",
  "password_confirmation": "string (Requerido)"
}
```

- Response (200 OK JSON / 302 Found): {"success": true, "message": "Contraseña actualizada exitosamente."}

3.7. Activar / Desactivar Usuario (Toggle State)

- Método y URL: PATCH /admin/users/{id}/toggle-state
- Response:
- 200 OK (JSON):

```
{
  "success": true,
  "message": "La cuenta de @usuario ha sido desactivada exitosamente.",
  "state": false
}
```

- 422 Unprocessable Entity: Error si el administrador intenta desactivar su propia cuenta.

3.8. Configuración del Sistema (Tipos de Dispositivos)

- Método y URL: GET /admin/configuracion
- Request (Query Params): search, status, sort_by, sort_direction, per_page, tab.
- Response (200 OK): Renderiza la vista admin.settings.index.

3.9. Registrar Tipo de Dispositivo

- Método y URL: POST /admin/configuracion/tipos-dispositivos
- Request:

```
{
  "name": "string (Requerido, max:50, único)",
  "status": "boolean (Opcional, default: true)"
}
```

- Response (201 Created JSON / 302 Found):

```
{
  "success": true,
  "message": "Tipo de dispositivo 'Tablet' registrado exitosamente.",
  "deviceType": { "id": 6, "name": "Tablet", "status": true }
}
```

3.10. Actualizar Tipo de Dispositivo

- Método y URL: PUT /admin/configuracion/tipos-dispositivos/{id}
- Request: {"name": "string (Requerido, max:50, único ignorando id)", "status": "boolean"}
- Response (200 OK JSON / 302 Found): Retorna el tipo de dispositivo actualizado.

3.11. Activar / Desactivar Tipo de Dispositivo

- Método y URL: PATCH /admin/configuracion/tipos-dispositivos/{id}/toggle-status
- Response (200 OK JSON / 302 Found):

```
{
  "success": true,
  "message": "El tipo de dispositivo 'Laptop' ha sido desactivado exitosamente.",
  "status": false
}
```

4. MÓDULO DE RECEPCIÓN E INVENTARIO (role:ADMINISTRADOR,RECEPCIONISTA, Prefijo /receptionist)

4.1. Panel de Inicio de Recepción (Ventas y Nuevo Ticket)

- Método y URL: GET /receptionist/reception
- Request (Query Params opcionales): search, category, stock_status, sort_by, sort_direction, per_page.
- Response (200 OK): Renderiza receptionist.home con catálogo de productos, listado de clientes registrados, técnicos activos y tipos de equipos.

4.2. Registrar Venta de Productos (POS y Kardex)

- Método y URL: POST /receptionist/sales
- Request (JSON / Form Data):

```
{  
  "payment_method": "string (Requerido: EFECTIVO | TARJETA | TRANSFERENCIA)",  
  "amount_received": "numeric (Opcional, min:0)",  
  "items": [  
    {  
      "barcode": "string (Requerido, existe en products)",  
      "quantity": "integer (Requerido, min:1)"  
    }  
  ]  
}
```

- Response (201 Created - JSON):

```
{  
  "success": true,  
  "message": "¡Venta #1 registrada exitosamente con comprobante emitido!",  
  "sale": {  
    "id": 1,  
    "total_sale": 230.00,  
    "sale_date": "20/09/2026 10:30"  
  }  
}
```

(Genera automáticamente las salidas en inventory_movements y los registros contables en financial_transactions).

4.3. Recepción de Dispositivo (Crear Ticket de Taller y QR)

- Método y URL: POST /receptionist/reception/dispositivos
- Request (Form Data / JSON):

```
{
```

```

"client_mode": "string (Requerido: 'new' o 'existing')",
"existing_client_id": "integer (Requerido si client_mode=existing)",
"client_name": "string (Requerido si client_mode=new, max:100)",
"client_lastname": "string (Requerido si client_mode=new, max:100)",
"client_phone": "string (Requerido si client_mode=new, max:20)",
"client_dpi": "string (Opcional, max:14)",
"device_type": "integer (Requerido, existe en device_types)",
"device_brand": "string (Requerido, max:50)",
"device_model": "string (Requerido, max:50)",
"device_serial": "string (Opcional, max:100)",
"device_password": "string (Opcional, max:100)",
"reported_issue": "string (Requerido, max:1000)",
"reception_notes": "string (Opcional, max:1000)",
"total_charged": "numeric (Requerido, min:0)",
"deposit": "numeric (Requerido, min:0, no mayor a total_charged)",
"id_user_technician": "integer (Opcional, ID de técnico activo)"
}

- Response:
- 201 Created (JSON / AJAX):

{
"success": true,
"message": "¡Ticket #10 registrado exitosamente para el cliente Carlos Gómez!",
"pdf_url": "http://localhost:8000/receptionist/tickets/10/pdf",
"tracking_url": "http://localhost:8000/seguimiento/550e8400-e29b-41d4-a716-446655440000",
"qr_svg": "<svg ...></svg>",
"ticket": {
"id": 10,
"qr_token": "550e8400-e29b-41d4-a716-446655440000",
"state": "Recibido",

```

```

"total_charged": 350.00,
"deposit": 100.00,
"remaining_balance": 250.00,
"client": "Carlos Gómez",
"device": "Samsung Galaxy S22",
"technician": "Nelson Ixcolín",
"intake_date": "20/09/2026 10:45"
}
}

```

- 302 Found: Redirige a /receptionist/reception con mensaje de confirmación.

4.4. Descarga de Ticket en PDF (Recepción)

- Método y URL: GET /receptionist/tickets/{id}/pdf
- Request: Parámetro de ruta id (entero).
- Response (200 OK - PDF): Descarga del comprobante con etiqueta para el equipo y ticket de retiro para el cliente.

4.5. Módulo de Inventario (Vista de Productos y Categorías)

- Método y URL: GET /receptionist/reception/inventory
- Request (Query Params): search, category, stock_status, status, category_search, category_status, per_page.
- Response (200 OK): Renderiza receptionist.inventory.index con tablas paginadas de productos y categorías.

4.6. Registrar Producto en Inventario

- Método y URL: POST /receptionist/reception/inventory/products
- Request (Multipart Form Data):

```

{
"bar_code": "string (Requerido, max:100, único)",
"name": "string (Requerido, max:150)",
"id_category": "integer (Requerido, existe en category_products)",

```

```

"description": "string (Opcional)",
"price": "numeric (Requerido, min:0.01)",
"stock": "integer (Requerido, min:0)",
"minium_stock": "integer (Requerido, min:1)",
"status": "boolean (Opcional, default: true)",
"image": "file/image (Opcional, max: 2048 KB)"
}

```

- Response (201 Created JSON / 302 Found): Retorna el objeto del producto creado e inserta automáticamente la primera entrada en Kardex si stock > 0.

4.7. Actualizar Producto en Inventario

- Método y URL: PUT /receptionist/reception/inventory/products/{barcode}
- Request:
- Parámetro de ruta: barcode (string, código de barras actual).
- Body: name, id_category, description, price, minium_stock, status, image.
- Response (200 OK JSON / 302 Found): Retorna el producto actualizado.

4.8. Activar / Desactivar Producto

- Método y URL: PATCH /receptionist/reception/inventory/products/{barcode}/toggle-status
- Response (200 OK JSON / 302 Found): Cambia el atributo status (borrado lógico).

4.9. Registrar Categoría de Productos

- Método y URL: POST /receptionist/reception/inventory/categorias
- Request:

```

{
"name": "string (Requerido, max:50, único)",
"description": "string (Opcional, max:255)",
"status": "boolean (Opcional, default: true)"
}

```

- Response (201 Created JSON / 302 Found): Retorna la categoría creada.

4.10. Actualizar Categoría de Productos

- Método y URL: PUT /receptionist/reception/inventory/categorias/{id}
- Request: {"name": "string (Requerido, max:50)", "description": "string (Opcional, max:255)"}
- Response (200 OK JSON / 302 Found): Retorna la categoría modificada.

4.11. Activar / Desactivar Categoría

- Método y URL: PATCH /receptionist/reception/inventory/categorias/{id}/toggle-status
- Response (200 OK JSON / 302 Found): Cambia el estado de la categoría.

4.12. Registrar Movimiento de Inventario Manual (Kardex)

- Método y URL: POST /receptionist/reception/inventory/movimientos
- Request:

```
{  
  "product_bar_code": "string (Requerido, existe en products)",  
  "movement_type": "string (Requerido: 'ENTRADA' | 'SALIDA' | 'AJUSTE')",  
  "quantity": "integer (Requerido, min:1 o min:0 si es AJUSTE)",  
  "reason": "string (Requerido, max:255)",  
  "notes": "string (Opcional, max:500)"  
}
```

- Response (201 Created JSON / 302 Found): Ajusta las existencias del producto y añade la transacción en la bitácora de movimientos.

4.13. Historial de Movimientos de Inventario (Kardex)

- Método y URL: GET /receptionist/reception/historiales
- Request (Query Params): search, movement_type, date_from, date_to, per_page.
- Response (200 OK): Renderiza receptionist.inventory.kardex.

4.14. Módulo de Entregas (Tickets Listos o Finalizados)

- Método y URL: GET /receptionist/reception/entregas
- Request (Query Params): search, state, date_from, date_to, per_page.
- Response (200 OK): Renderiza receptionist.deliveries.index.

4.15. Procesar Entrega y Liquidación de Dispositivo

- Método y URL: POST /receptionist/reception/entregas/{id}/entregar
- Request:

- Parámetro de ruta: id (ID del ticket).

- Body:

```
{  
  "return_date": "date/datetime (Requerido)",  
  "amount_to_pay": "numeric (Requerido, min:0)",  
  "payment_method": "string (Requerido: 'EFECTIVO' | 'TARJETA' | 'TRANSFERENCIA')",  
  "delivery_notes": "string (Opcional, max:500)"  
}
```

- Response (200 OK JSON / 302 Found): Cambia el estado del ticket a Entregado, liquida el saldo restante y registra el ingreso financiero.

5. MÓDULO TÉCNICO (role:ADMINISTRADOR,TECNICO, PREFIJO /technician)

5.1. Panel de Inicio del Técnico

Método y URL: GET /technician/technician

Request (Query Params opcionales):

- search: búsqueda por código de ticket, cliente o falla.
- state: filtro por estado (Recibido, Diagnóstico, Reparación, Finalizado).
- date_from, date_to: rango de fechas.
- technician_id: filtro por técnico específico (sólo visible para Administradores).
- per_page: cantidad de tickets por página.

Response (200 OK): Renderiza technician.home con los tickets asignados y métricas del técnico.

5.2. Avanzar Estado del Ticket

Método y URL: PATCH /technician/tickets/{id}/state

Request:

- Parámetro de ruta: id (ID del ticket).

- Body (JSON o Form Data):

```
{  
  "state": "string (Requerido: 'Diagnóstico' | 'Reparación' | 'Finalizado')",  
  "technical_diagnosis": "string (Opcional, max:1000)"  
}
```

Response:

- 200 OK (JSON):

```
{  
  "success": true,  
  "message": "¡Ticket #12 avanzado con éxito al estado 'Reparación'!",  
  "ticket": { "id": 12, "state": "Reparación", "...": "..." }  
}
```

- 403 Forbidden: Si el técnico intenta modificar un ticket que no tiene asignado.

- 422 Unprocessable Entity: Si intenta retroceder el estado o saltarse fases de la reparación.

5.3. Registrar Nota Técnica en Bitácora del Ticket

Método y URL: POST /technician/tickets/{id}/notes

Request:

```
{  
  "note": "string (Requerido, min:3, max:1000)"  
}
```

Response (201 Created - JSON):

```
{  
  "success": true,  
  "message": "Nota técnica agregada correctamente al ticket #12.",  
  "note": {  
    "id": 4,  
    "id_ticket": 12,
```

```
"note": "Se reemplazó el módulo de carga y se limpiaron los conectores.",  
"created_at": "20/09/2026 11:15",  
"user": { "id": 3, "name": "Técnico Especialista" }  
}  
}
```

5.4. Listar Notas de un Ticket (Historial de Bitácora)

- Método y URL: GET /technician/tickets/{id}/notes
- Response (200 OK - JSON):

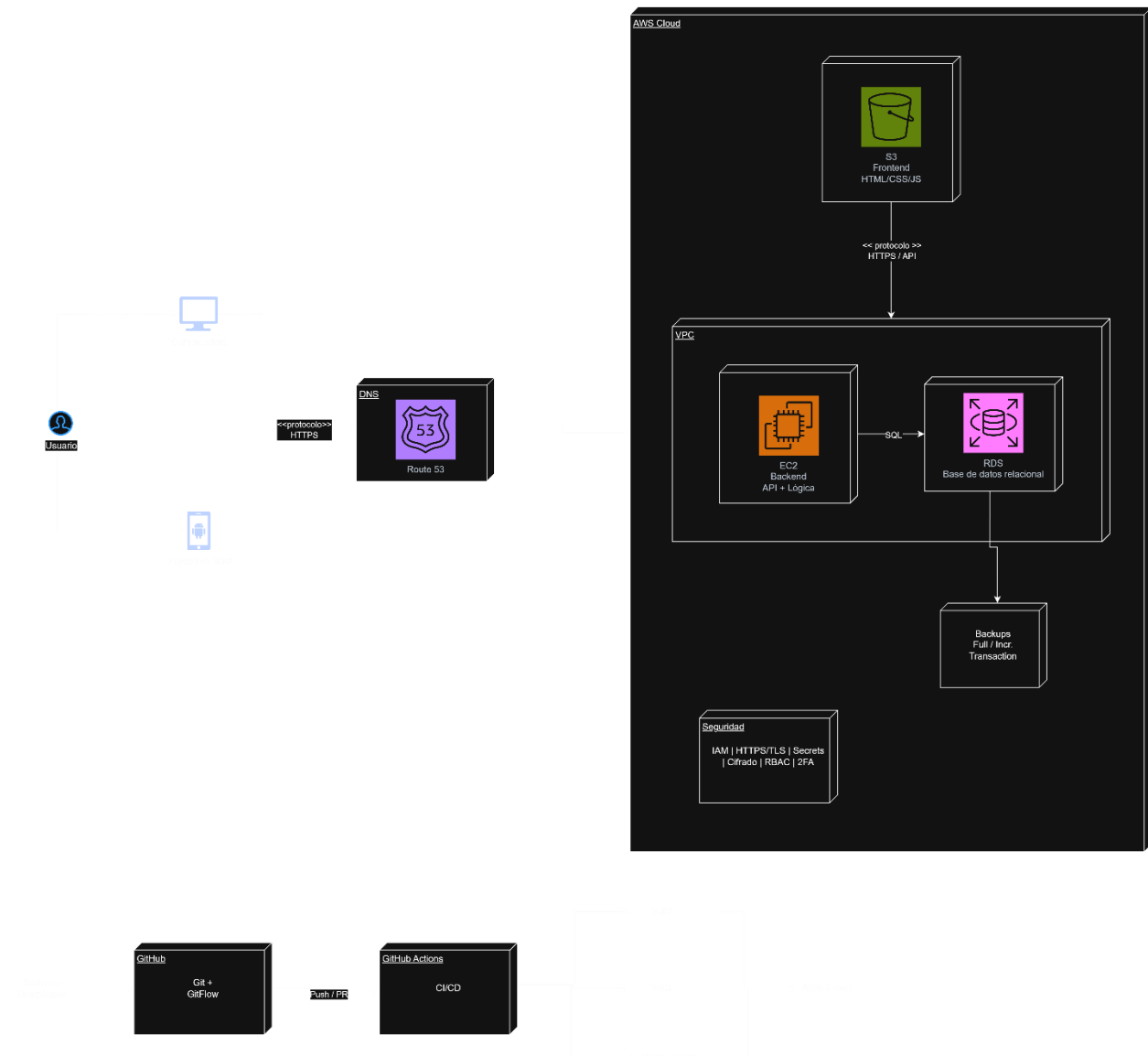
```
{  
  "success": true,  
  "notes": [  
    {  
      "id": 1,  
      "note": "Revisión inicial del flex de pantalla",  
      "created_at": "2026-09-20T11:00:00.000000Z",  
      "user": { "id": 3, "name": "Nelson Ixcolín" }  
    }  
  ]  
}
```

Diagramas

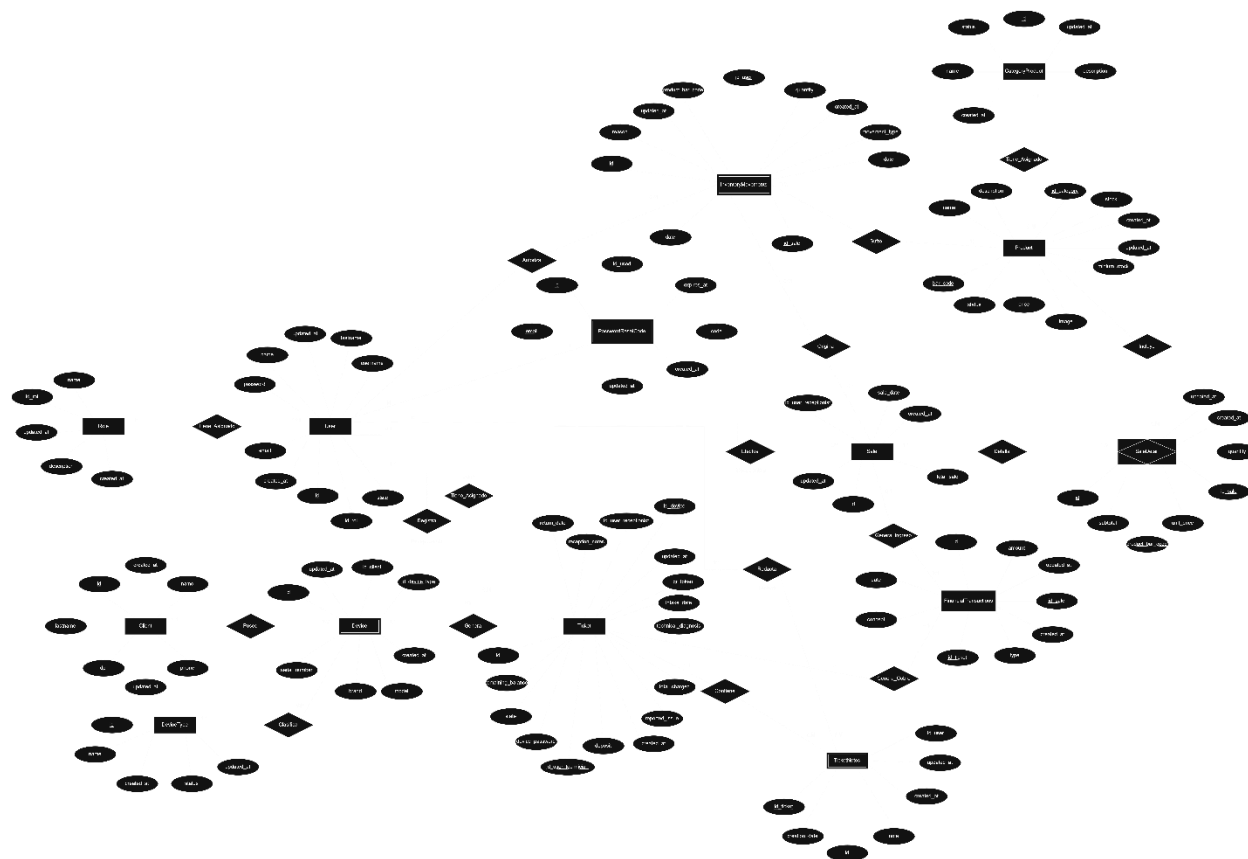
Diagrama de Clases



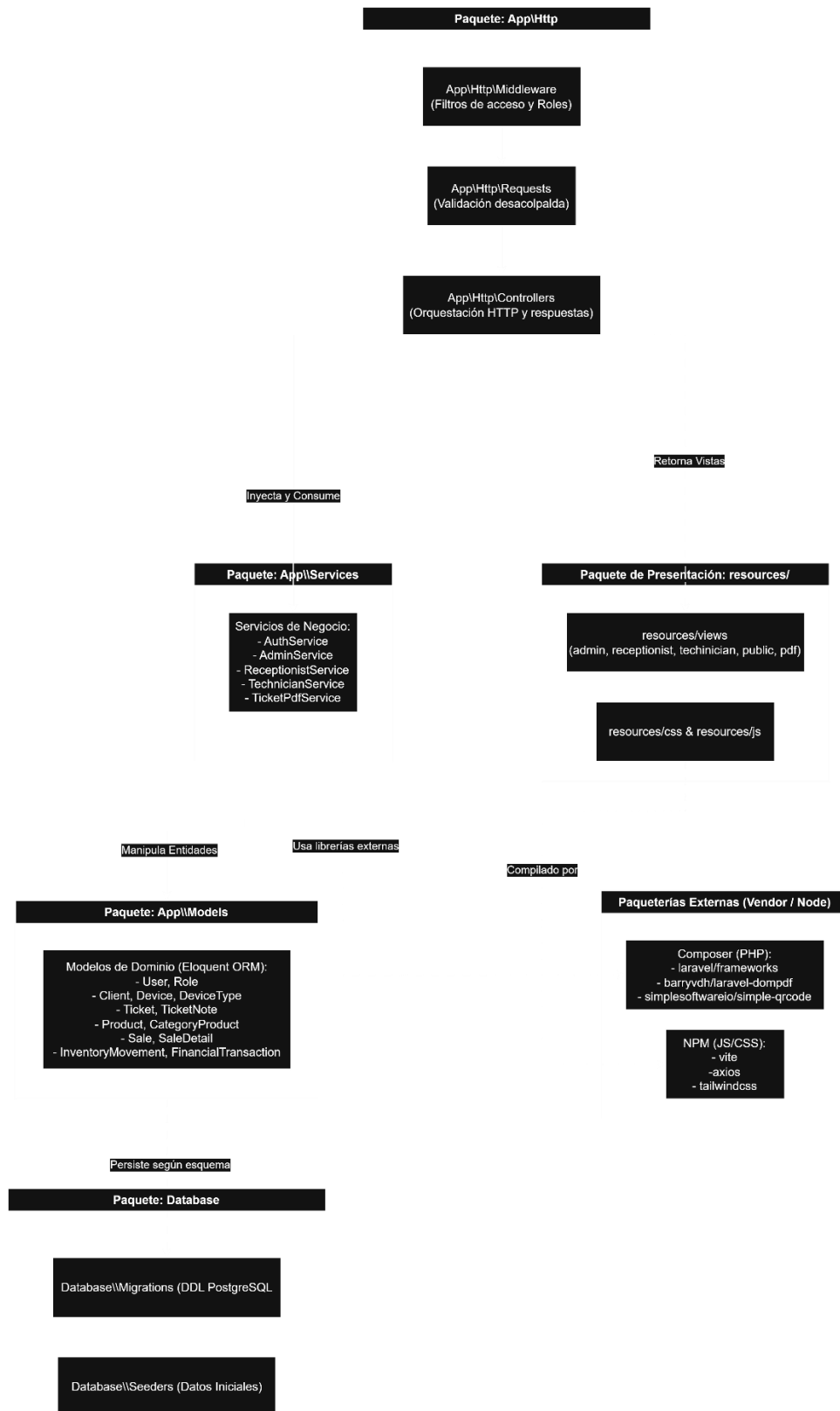
Diagrama de Arquitectura e Infraestructura



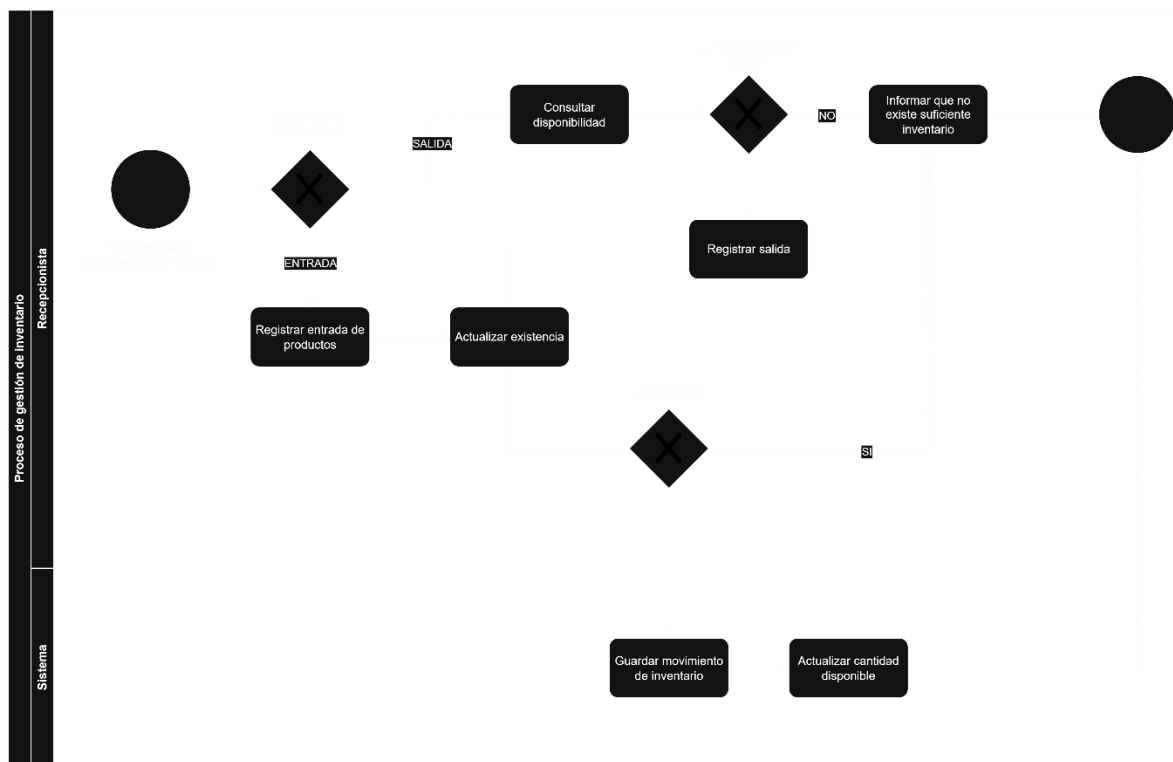
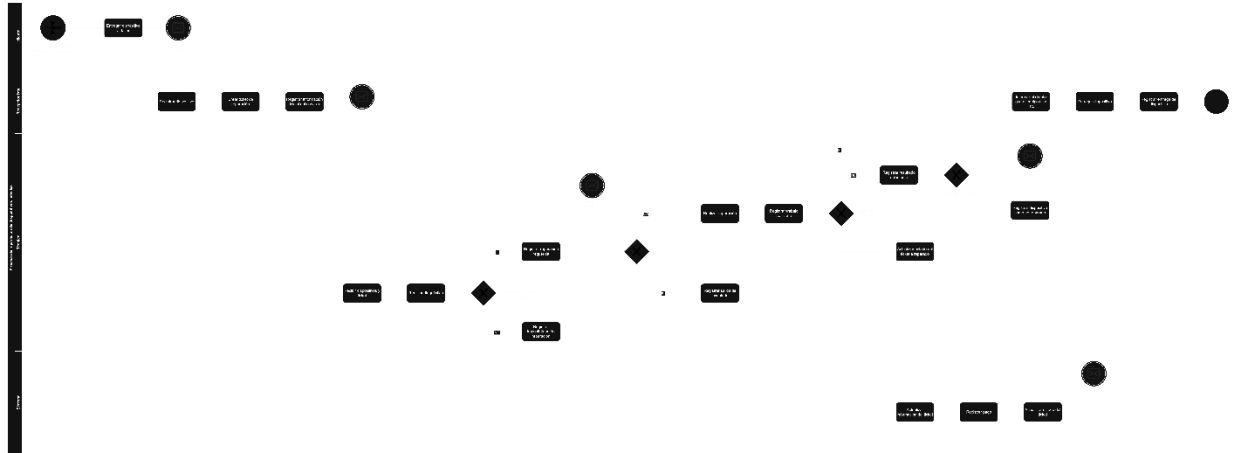
Modelo Entidad-Relación (E/R)



Estructura de Paqueterías



Modelos de Procesos Técnicos



Historias de Usuarios

Épica 1: Autenticación, Seguridad y Gestión de Usuarios

SCRUM-01: Inicio de Sesión Seguro con RBAC y Username

Historia de Usuario:

Como empleado del negocio (Administrador, Recepcionista o Técnico),

Quiero iniciar sesión utilizando mi nombre de usuario (username) y contraseña cifrada,

Para acceder al sistema de forma protegida y visualizar únicamente las funciones permitidas para mi rol.

Criterios de Aceptación (DoD):

- Validación estricta de credenciales contra la base de datos PostgreSQL utilizando hashing seguro (Bcrypt/Argon2).
- Redirección automática a la vista inicial correspondiente según el rol asignado (Administrador, Recepcionista o Técnico).
- Cierre e invalidación completa de la sesión al presionar "Cerrar Sesión".
- Mensaje claro de error ante credenciales inválidas sin revelar qué dato específico falló.
- Bloqueo de rutas y vistas protegidas ante accesos no autenticados o no autorizados mediante middlewares en Laravel.

SCRUM-02: Creación y Gestión Administrativa de Usuarios

Historia de Usuario:

Como Administrador,

Quiero registrar, modificar, activar y desactivar cuentas de trabajadores desde un panel centralizado,

Para controlar rigurosamente quién opera la plataforma sin permitir registros públicos.

Criterios de Aceptación (DoD):

- Acceso restringido exclusivamente a usuarios con rol Administrador.
- Formulario de creación de usuario con validación de campos obligatorios: username, name, email, contraseña y selección de rol (id_rol).
- Restricción total de registros públicos o enlaces abiertos de autoregistro.
- Funcionalidad para activar o desactivar usuarios mediante un botón de alternancia sin eliminar el registro físico.
- El listado de usuarios muestra claramente el estado (Activo/Inactivo) y rol de cada cuenta.

SCRUM-03: Recuperación de Contraseña y Autenticación en Dos Factores (2FA)

Historia de Usuario:

Como usuario del sistema,

Quiero restablecer mi contraseña mediante un código temporal enviado a mi correo y contar con verificación en dos factores (2FA),

Para recuperar el acceso a mi cuenta y proteger las acciones críticas del sistema.

Criterios de Aceptación (DoD):

- Formulario público de recuperación que solicita el correo electrónico registrado.
- Envío de correo electrónico transaccional con un código numérico temporal o token con tiempo de expiración.
- Verificación correcta del código previo a permitir la definición de la nueva contraseña.
- Modal interno para cambio de contraseña desde la vista de perfil.
- Las contraseñas se almacenan cifradas y bajo ninguna circunstancia se visualizan en texto plano ni son accesibles por otros usuarios.

Épica 2: Gestión del Flujo de Tickets de Reparación

SCRUM-04: Recepción de Dispositivos y Creación de Tickets con Multidispositivo

Historia de Usuario:

Como Recepcionista,

Quiero registrar un nuevo servicio seleccionando el tipo de equipo, cliente, fallas, técnico y fotografía del dispositivo,

Para generar un ticket de reparación con identificador único y comenzar el ciclo de trabajo.

Criterios de Aceptación (DoD):

- Selección dinámica del tipo de dispositivo a través del catálogo DeviceType (Celular, Tablet, Consola, Audífonos, etc.).
- Registro o selección del cliente (Client) y descripción detallada del problema o motivo de ingreso.
- Carga obligatoria o adjunto de al menos una fotografía del estado físico inicial del equipo.
- Asignación obligatoria del técnico responsable (id_user_technician) desde un listado filtrado por rol Técnico.
- Creación del ticket en estado inicial "Recibido" con asignación de fecha y hora automática.

SCRUM-05: Bandeja Técnica, Diagnóstico y Actualización de Bitácora

Historia de Usuario:

Como Técnico,

Quiero consultar mi bandeja de dispositivos asignados, cambiar su estado de trabajo y registrar notas técnicas con repuestos utilizados,

Para llevar la trazabilidad exacta de las reparaciones bajo mi responsabilidad.

Criterios de Aceptación (DoD):

- La pantalla "Dispositivos Para Trabajar" solo muestra al técnico los tickets que tienen asignado su ID de usuario.
- Modal de actualización de estado para transicionar el ticket entre las etapas: Diagnóstico, En Reparación y Finalizado.
- Modal de bitácora técnica para ingresar comentarios, observaciones de fallas encontradas y costos de repuestos asociados.
- Historial cronológico de cambios de estado y notas registradas visible en la consulta detallada del ticket.

SCRUM-06: Consulta Pública y Seguimiento de Reparación mediante Código QR

Historia de Usuario:

Como Cliente de Cellx,

Quiero escanear un código QR con la cámara de mi teléfono móvil,

Para verificar en tiempo real el avance y estado de mi reparación sin tener que llamar al local ni iniciar sesión.

Criterios de Aceptación (DoD):

- Generación de un token criptográfico único (qr_token de tipo UUID) al crearse el ticket.
- Generación del código QR imprimible o visualizable en el comprobante de recepción que apunta a /tracking/{qr_token}.
- Vista pública de solo lectura que presenta el modelo del equipo, fecha de ingreso, estado actual y observaciones públicas.
- Restricción de acceso a datos sensibles (no se exponen montos internos, contraseñas ni datos personales de otros clientes).
- Manejo de error 404 amigable si el token consultado no existe o es inválido.

SCRUM-07: Entrega de Equipo, Cobro de Mano de Obra y Cierre de Ticket

Historia de Usuario:

Como Recepcionista,

Quiero marcar el equipo como entregado al cliente registrando el monto cobrado por el servicio,

Para cerrar formalmente el ticket y generar el ingreso monetario correspondiente.

Criterios de Aceptación (DoD):

- El botón de entrega solo se habilita para tickets que se encuentren en estado "Finalizado" por el técnico.
- Formulario modal que solicita el monto total cobrado por el servicio de reparación.
- Al confirmar, el ticket pasa a estado "Entregado" y se registra la fecha y hora de entrega.
- Creación automática de un registro en la tabla de transacciones financieras clasificado como Ingreso con referencia al ticket.

Épica 3: Control y Gestión de Inventario

SCRUM-08: Catálogo de Accesorios y Semáforo Visual de Existencias

Historia de Usuario:

Como Recepcionista o Administrador,

Quiero consultar el listado de accesorios clasificado por colores según la disponibilidad de stock,

Para identificar rápidamente los productos que están por agotarse y gestionar su catálogo.

Criterios de Aceptación (DoD):

- Operaciones CRUD para accesorios (Nombre, código de barra `product_bar_code`, categoría, precio y fotografía).
- Semáforo de stock condicional por filas o badges con los colores oficiales:
 - o **Verde (#16A34A)**: Existencias normales por encima del umbral.
 - o **Amarillo (#D97706)**: Existencias en advertencia.
 - o **Rojo (#DC2626)**: Existencias críticas o agotadas.
- Barra de búsqueda con filtros reactivos por nombre del producto y categoría.

SCRUM-09: Control de Entradas/Salidas y Alerta de Stock Mínimo

Historia de Usuario:

Como Administrador,

Quiero registrar entradas o salidas manuales de inventario y configurar un umbral mínimo por producto (`minium_stock`),

Para mantener auditadas las existencias y recibir notificaciones cuando se requiera reabastecimiento.

Criterios de Aceptación (DoD):

- Registro de movimientos de inventario (`InventoryMovements`) indicando tipo (Entrada/Salida), cantidad, motivo y usuario responsable.
- Actualización en tiempo real del stock general del producto tras cada movimiento.
- Configuración del campo `minium_stock` en el formulario del producto.
- Activación de alerta visual inmediata cuando el stock sea menor o igual al umbral fijado.
- Disparo de evento y notificación interna/correo al Administrador advirtiendo la necesidad de reabastecimiento.

Épica 4: Control Financiero Básico y Ventas

SCRUM-10: Venta de Accesorios Maestro-Detalle de Mostrador

Historia de Usuario:

Como Recepcionista,

Quiero procesar la venta de uno o varios accesorios en una misma transacción calculando subtotales,

Para cobrar al cliente de inmediato y descontar las cantidades vendidas del stock.

Criterios de Aceptación (DoD):

- Interfaz de venta rápida que permite agregar múltiples líneas de productos mediante búsqueda o código de barras.
- Cálculo dinámico de subtotales por producto y monto total general de la venta.
- Validación obligatoria de existencia suficiente antes de autorizar la transacción.
- Al confirmar la venta: se crea el registro en sales, sus detalles en sale_details, se decrementa el inventario y se genera un movimiento financiero de Ingreso.

SCRUM-11: Registro y Clasificación de Egresos Operativos

Historia de Usuario:

Como Administrador o Recepcionista,

Quiero registrar los gastos del negocio y compras de mercadería categorizadas,

Para mantener un historial balanceado de las salidas de dinero de Cellx.

Criterios de Aceptación (DoD):

- Formulario de egreso que solicita: monto, concepto descriptivo, categoría del gasto y fecha.
- Validación para evitar montos negativos o campos vacíos.
- Registro en la tabla de transacciones financieras marcado como tipo "Egreso".
- Tabla general de movimientos financieros con filtros por tipo (Ingreso/Egreso) y rangos de fechas.

Épica 5: Reportería, Métricas y Dashboard Ejecutivo

SCRUM-12: Dashboard Gerencial con Métricas Operativas y Financieras

Historia de Usuario:

Como Administrador,

Quiero visualizar un panel de control con indicadores clave (KPIs) y gráficos del taller, ventas e inventario,

Para monitorear el desempeño global del negocio y tomar decisiones estratégicas basadas en datos.

Criterios de Aceptación (DoD):

- Tarjetas de métricas principales: Total de ingresos acumulados en el período, balance neto (ingresos menos egresos) y cantidad de usuarios activos/inactivos.
- Gráfico o resumen de tickets desglosados por estado (Recibido, Diagnóstico, En Reparación, Finalizado, Entregado).
- Sección destacada con los accesorios con mayor y menor cantidad de ventas.
- Filtro temporal para visualizar los datos por día, semana, mes o rango personalizado.

- Acceso restringido exclusivamente a usuarios con rol Administrador.

SCRUM-13: Módulo de Reportes con Exportación Obligatoria a PDF y Excel

Historia de Usuario:

Como Administrador,

Quiero generar reportes filtrados de reparaciones, ventas y finanzas, y exportarlos en formatos PDF y Excel (.xlsx),

Para realizar auditorías contables externas y archivar el historial de la empresa.

Criterios de Aceptación (DoD):

- Tabla de reportes con filtros por rango de fechas, categorías de productos y estados de tickets.
- Botón "Exportar a PDF" que genera y descarga un documento formateado con encabezado institucional y datos filtrados (barryvdh/laravel-dompdf).
- Botón "Exportar a Excel" que descarga un archivo .xlsx estructurado y procesable (maatwebsite/excel).
- Trazabilidad que incluye la fecha de emisión del reporte y el usuario que lo generó.
- Acceso restringido exclusivamente al rol Administrador.

Casos de Uso

CU-01: Autenticación e Inicio de Sesión

Atributo	Detalle
Identificador	CU-01
Nombre	Autenticación e Inicio de Sesión
Actores	Actor Principal: Empleado (Administrador, Recepcionista o Técnico). Actor Secundario: Sistema Cellx (Servicio de Autenticación y PostgreSQL).
Propósito	Validar la identidad del empleado para otorgar acceso seguro y redirigir al panel correspondiente según su rol (RBAC).
Tipo	Primario, Esencial.
Precondiciones	El usuario debe estar previamente registrado por el Administrador y tener el estado activo (state = true). La aplicación debe estar accesible vía HTTPS/LTS.
Postcondiciones	Éxito: Sesión HTTP iniciada en PostgreSQL (sessions), token CSRF regenerado y usuario situado en su panel correspondiente. Fallo: Acceso denegado, sesión no creada y registro del intento fallido en logs.

Flujo Principal de Eventos

Paso	Acción del Actor	Respuestas del Sistema
1	El usuario ingresa a la URL raíz /login.	Presenta la interfaz de inicio de sesión con el logotipo de Cellx, campos de credenciales y botón de acción.
2	Ingresa su nombre de usuario, contraseña y presiona “Iniciar Sesión”.	Captura la petición POST/login y ejecuta las validaciones de LoginRequest.
3		Verifica que el usuario exista en la tabla users.
4		Compara la contraseña suministrada contra el hash Bcrypt almacenado en la base de datos.
5		Valida que la cuenta esté activa (state = true).
6		Regenera el ID de sesión para prevenir ataques de fijación de sesión y guarda el registro en la tabla sessions.
7		Evalúa el rol asignado (id_rol) a través de AuthService y redirige el dashboard correspondiente: - ADMINISTRADOR -> /admin/home - RECEPCIONISTA -> /receptionist/reception - TECNICO -> /technician/technician

Flujos Alternativos y Excepciones

- **3a. Usuario no encontrado o contraseña incorrecta:**
 - o El sistema no especifica cuál de los dos datos falló por seguridad.

- Redirige al formulario /login manteniendo el valor del nombre de usuario y mostrando el mensaje “Las credenciales proporcionadas no coinciden con nuestros registros.”
- **5a. Cuenta desactivada (state = false):**
 - El sistema rechaza la autenticación de inmediato.
 - Muestra la alerta de error. “Esta cuenta se encuentra inactiva. Comuníquese con el Administrador.”
- **2a. Campos vacíos o formato inválido:**
 - El validador LoginRequest falla (HTTP 422). Retorna los mensajes de validación en español al pie de cada campo.

CU-02: Gestión de Cuentas de Usuario

Atributo	Detalle
Identificador	CU-02
Nombre	Gestión de Cuentas de Usuario y Asignación de Roles
Actores	Actor Principal: Administrador. Actor Secundario: PostgreSQL (users, roles).
Propósito	Registrar nuevos empleados, modificar datos, restablecer contraseñas y activar/desactivar accesos.
Tipo	Primario.
Precondiciones	El administrador debe haber iniciado sesión con rol ADMINISTRADOR.
Postcondiciones	Se persiste o actualiza la cuenta en la tabla users conservando la integridad referencial histórica.

Flujo Principal de Eventos

Paso	Acción del Actor	Respuestas del Sistema
1	El Administrador accede al menú “Gestión de Usuarios” (/admin/users).	Muestra la vista con métricas de cuentas (activas, inactivas, total) y la tabla paginada de usuarios con buscador y filtros.
2	Presiona el botón “Registrar Usuario”.	Despliega un modal interactivo con el formulario de registro.
3	Ingresa nombre, apellido, username, correo electrónico, selecciona el rol, contraseña y confirmación. Presiona “Guardar”.	Envía los datos vía POST /admin/users/create . Valida unicidad de correo y username con StoreUserRequest.
4		Encripta la contraseña mediante Bcrypt y crea el registro en la tabla users con state = true.
5		Cierra el modal, actualiza la tabla de usuarios vía AJAX/SSR y emite notificación de éxito con badge verde.

Flujos Alternativos (Desactivación de Usuario)

- **3a. El Administrador solicita cambiar el estado de un usuario (Toggle State):**
 - Presiona el botón de acción “Desactivar” en la fila del empleado.
 - El sistema verifica que el usuario objetivo no sea el mismo Administrador en sesión.
 - Si es otro usuario: actualiza users.state = false (borrado lógico), impidiendo su futuro inicio de sesión sin eliminar sus ventas o tickets asociados.
 - Si intenta auto-desactivarse: bloquea la acción (HTTP 422) y muestra: “No puedes desactivar tu propia cuenta de administrador en sesión.”

CU-03: Admisión de Dispositivo y Emisión de Ticket con Código QR

Atributo	Detalle
Identificador	CU – 03
Nombre	Admisión de Dispositivo Móvil y Emisión de Orden de Servicio
Actores	Actor principal: Recepcionista. Actores Secundarios: Cliente, Técnico Asignado, TicketPdfService.
Propósito	Formalizar el ingreso de un equipo averiado, registrar la falla, cobrar anticipos si aplica, generar el token QR y emitir el comprobante físico.
Tipo	Primario, Transaccional.
Precondiciones	El recepcionista debe estar autenticado. Deben existir tipos de dispositivos registrados en el catálogo.
Postcondiciones	Se crea el cliente (si es nuevo), el dispositivo en devices, el ticket en tickets en estado Recibido, se registra el anticipo en financial_transactions (si aplica) y se genera el archivo PDF con código QR.

Flujo Principal de Eventos

Paso	Acción del Actor	Respuestas del Sistema
1	El Recepcionista pulsa “Registrar Dispositivo” en su panel principal.	Despliega el modal de admisión de equipo.
2	Selecciona si el cliente es nuevo o del directorio existente.	Habilita los campos de cliente (nombre, apellido, teléfono / WhatsApp, DPI) o el buscador de clientes existentes.
3	Selecciona el tipo de equipo (Teléfono, Tablet, etc.), digita marca, modelo, serie/IMEI, contraseña/patrón de desbloqueo, falla reportada y observaciones estéticas iniciales.	Valida en tiempo real la obligatoriedad de los datos técnicos.
4	Define el costo pactado del trabajo (total_charged), el monto del anticipo (deposit) y selecciona opcionalmente un técnico responsable de la lista de activos.	Comprueba que el anticipo no sea negativo ni superior al costo total (lte:total_charged).
5	Presiona “Registrar Orden”.	Inicia una transacción atómica de base de datos (DB::transaction).
6		Si el cliente es nuevo, lo inserta en clients ; registra el dispositivo en devices.

7		Genera un token criptográfico único UUID v4 (qr_token).
8		Inserta el registro en tickets con estado inicial “Recibido” y fecha/hora del servidor.
9		Si deposit > 0, inserta automáticamente un asiento contable en financial_transactions con tipo “INGRESO”.
10		Confirma la transacción (COMMIT).
11		TicketPdfService genera el código QR vectorial SVG apuntando a /seguimiento/{qr_token} y compila el PDF de doble tablón.
12		Retorna HTTP 201 Creted con el payload del ticket y abre la descarga automática del archivo ticket-celix-TK-{id}.pdf .

Flujos Alternativos

- **4a. El cliente no deja anticipo:**
 - o El recepcionista ingresa deposit = 0. El sistema omite la creación de la transacción financiera y registra el ticket con el saldo total pendiente en la columna virtual remaining_balance.
- **5a. Falla de validación en los datos:**
 - o El FormRequest StoreDeviceIntakeRequest rechaza la solicitud (HTTP 422) y resalta los campos con error sin alterar la base de datos.

CU-04: Diagnóstico y Actualización de Estado Técnico en Taller

Atributo	Detalle
Identificador	CU-04
Nombre	Diagnóstico Técnico, Bitácora y Avance de Estado de Reparación.
Actores	Actor Principal: Técnico. Actor Secundario: Administrador (supervisión).
Propósito	Gestionar la orden en laboratorio, asentar el diagnóstico técnico, registrar notas cronológicas y avanzar el ciclo de vida del trabajo.
Tipo	Primario
Precondiciones	El ticket debe encontrarse en estado Recibido, Diagnóstico o Reparación y estar asignado al técnico en sesión (o usuario Administrador).
Postcondiciones	Estado del ticket actualizado en base de datos; inserción de nota técnica auditada en ticket_notes . El avance es irreversible.

Flujo Principal de Eventos

Paso	Acción del Actor	Respuestas del Sistema
------	------------------	------------------------

1	El Técnico ingresa a su panel (/technician/technician).	Carga los tickets asignados con sus badges de estado, datos del equipo y motivo de ingreso.
2	Presiona el botón “Diagnóstico / Avanzar Estado” en el ticket desado.	Despliega modal con el historial del ticket y los estados válidos hacia los que puede progresar (getNextAllowedStates).
3	Selecciona el nuevo estado (Diagnóstico, Reparación o Finalizado) e ingresa la justificación o diagnóstico técnico.	Verifica mediante la regla STATE_ORDER que el peso del nuevo estado sea estrictamente mayor al actual (newWeight > currentWeight).
4	Presiona “Actualizar Estado”.	Envía la solicitud PATCH /technician/tickets/{id}/state.
5		Actualiza el campo tickets.state y tickets.technical_diagnosis.
6		Crea una entrada automática en ticket_notes vinculando el ID del ticket, ID del técnico, contenido de la acción y timestamp.
7		Retorna confirmación HTTP 200 JSON y actualiza el contador de KPIs del taller.

Flujos Alternativos y Excepciones

- **3a. Intento de retroceso de estado (ej. De Reparación a Diagnóstico):**
 - o El sistema bloque la operación y lanza ValidationException : “El flujo técnico es irreversible y no se permite retroceder el estado.”
- **3b. El técnico intenta marcar como “Entregado”:**
 - o El sistema rechaza la petición: “El estado ‘Entregado’ únicamente puede ser asignado por Recepción al momento de liquidar y entregar el equipo.”
- **3c. Adición de nota sin cambio de estado:**
 - o El técnico pulsa “Notas de Seguimiento” (POST /technician/tickets/{id}/notes). El sistema registra la observación técnica en la bitácora sin alterar el estado actual.

CU-05: Entrega y Liquidación de Dispositivo Reparado

Atributo	Detalle
Identificador	CU-05
Nombre	Entrega de Dispositivos y Liquidación de Saldo
Actores	Actor Principal: Recepcionista. Actores Secundarios: Cliente, Sistema Financiero (financial_transactions).
Propósito	Entregar el equipo reparado al cliente, cobrar el saldo restante, registrar el movimiento contable y cerrar el ticket definitivamente.
Tipo	Primario, Transaccional
Precondiciones	El ticket debe encontrarse en estado “Finalizado”.
Postcondiciones	El ticket pasa a “Entregado”, se fija return_date = NOW(), se crea el asiento de “INGRESO” por el saldo cobrado y el ticket queda cerrado para modificaciones técnicas.

Flujo Principal de Eventos

Paso	Acción del Actor	Respuestas del Sistema
1	El cliente acude al taller. El Recepcionista accede al módulo de Entregas (/receptionist/entregas).	Muestra la lista de dispositivos finalizados listos para entrega con el saldo pendiente calculado (remaining_balance).
2	El Recepcionista localiza la orden por folio, cliente o serie y presiona “Procesar Entrega”.	Abre el modal de liquidación mostrando: costo total, anticipo pagado y saldo a cobrar en Quetzales.
3	Solicita el pago al cliente, selecciona el método utilizado (EFECTIVO, TARJETA o TRANSFERENCIA) e ingresa observaciones de entrega si las hay.	Valida que el monto liquidado sea consistente con el saldo pendiente mediante DeliverTicketRequest.
4	Presiona “Confirmar Entrega y Liquidación”.	Inicia transacción atómica en PostgreSQL.
5		Actualiza tickets.state = “Entregado” y estampa fecha/hora en tickets.return_date.
6		Si el saldo ccobrado es > 0, inserta el registro en financial_transactions con tipo “INGRESO” , asociando el ID del ticket y la descripción del equipo/cliente.
7		Confirma la transacción (COMMIT).
8		Notifica al usuario con alerta verde, remueve el ticket de la bandeja de pendiente y emite el comprobante final.

CU-06: Venta de Mostrador (POS) y Control Automático de Kardex

Atributo	Detalle
Identificador	CU-06
Nombre	Venta de Mostrador con Descuento Automático en Kardex
Actores	Actor Principal: Recepcionista. Actor Secundario: Cliente, Sistema e Inventario y Finanzas.
Propósito	Registra la venta de uno o varios accesorios, cobrarlos en mostrador, actualizar el stock físico y generar la bitácora contable y de Kardex.
Tipo	Primario Transaccional ACID.
Precondiciones	El Recepcionista debe estar autenticado. Los productos deben existir y tener estado activo (status = true).
Postcondiciones	Se crea la cabecera en sales , ítems en sale_details , se decrementa el stock en prodcuts , se registra salida en inventory_movements y el cobro de financial_transactions.

Flujo Principal de Eventos

Paso	Acción del Actor	Respuestas del Sistema
------	------------------	------------------------

1	El cliente solicita comprar accesorios. El Recepcionista pulsa “Registrar Venta” en el panel de inicio.	Despliega la interfaz modal de Punto de Venta (POS).
2	Busca los productos por código de barras o nombre y selecciona las cantidades deseadas.	Agrega los ítems al carrito, calculando dinámicamente subtotales y total generado en Quetzales.
3	Selecciona la forma de pago (EFECTIVO, TARJETA, TRANSFERENCIA), ingresa el monto recibido y presiona “Completar Venta”.	Envía payload JSON vía POST /receptionist/sales . Ejecuta StoreSaleRequest.
4		Abre una transacción de bse de datos (DB::transaction).
5		Aplica un bloque pesimista (FOR UPDATE) sobre las filas de los productos seleccionados para prevenir sobreventa por concurrencia.
6		Verifica que para cada ítem: producto.stock >= cantidad_solicitada .
7		Inserta la cabecera en sales con la fecha y monto consolidado verificado.
8		Por cada producto: inserta en sale_details , descuenta el inventario (stock = stock – cantidad) e inserta una fila en inventory_movements con tipo “SALIDA” y motivo “Venta #ID” .
9		Inserta el cobro total en financial_transactions con tipo “INGRESO”.
10		Confirma la transacción (COMMIT).
11		Responde HTTP 201 Created con el detalle de la venta, actualiza el catálogo visual (cambiando los badges de color si el stock llegó a nivel bajo o agotado) y emite el recibo de compra.

Flujos Alternativos

- **6a. Stock insuficiente en uno o más productos:**
 - La transacción ejecuta ROLLBACK inmediato.
 - Retorna HTTP 422 indicando el producto y las unidades disponibles reales.
 - Muestra alerta en modal: “Stock insuficiente para ‘{producto}’. Disponibles: X, Solicitadas: Y.”

CU-07: Consulta Pública de Estado Mediante Código QR

Atributo	Detalle
Identificador	CU-07
Nombre	Seguimiento Público de Reparación vía Código QR
Actores	Actor Principal: Cliente (Visitante externo sin credenciales).

	Actor Secundario: Sistema Cellx
Propósito	Permitir al cliente consultar en cualquier momento el avance en taller de su equipo escaneando el código QR de su boleta.
Tipo	Secundario, Consulta Pública.
Precondiciones	El cliente debe tener el comprobante impreso o digital con el código QR y conexión a internet.
Postcondiciones	Se presenta la vista pública responsiva con el estado actual del dispositivo sin exponer contraseñas ni datos sensibles de otros clientes.

Flujo Principal de Eventos

Paso	Acción del Actor	Respuestas del Sistema
1	El cliente escanea el código QR con su smartphone o accede al URL /seguimiento/{token} .	El enrutador captura la petición pública GET /seguimiento/{token} .
2		Valida que el parámetro {token} cumpla con la estructura de UUID v4.
3		Consulta en tickets por qr_token == token , cargando las relaciones del dispositivo, tipo y notas públicas.
4		Calcula la posición en el componente visual Stepper (1. Recibido -> 2. Diagnóstico -> 3. Reparación -> 4. Finalizado -> 5. Entregado).
5		Renderiza la vista public.ticket-tracking adaptada para móviles, mostrando marca, modelo, fecha de admisión, diagnóstico formal y notas de avance del técnico.
6	El cliente puede presionar el botón “Descargar Comprobante PDF”.	Descarga directamente el PDF del ticket sin requerir contraseña.

Flujos Alternativos

- **2a/3a. Token no válido o inexistente:**
 - o Si el UUID no existe o la cadena está truncada, el sistema aborta con HTTP 404 y renderiza la pantalla amigable public.not-found: “El ticket de servicio solicitado no fue encontrado o el enlace es incorrecto.”

CU-08: Consulta de Dashboard y Exportación de Reportes

Atributo	Detalle
Identificador	CU-08
Nombre	Consulta de Dashboard Gerencial y Exportación de Reportes (PDF/Excel)
Actores	Actor Principal: Administrador. Actor Secundario: Motor DomPDF, Generador Excel
Propósito	Analizar métricas operativas del taller, ventas y finanzas, y exportar reportes analíticos oficiales.
Tipo	Primario

Precondiciones	El usuario debe estar autenticado con el rol ADMINISTRADOR.
Postcondiciones	Visualización de indicadores en pantalla y descarga del archivo .pdf o .xlsx solicitado.

Flujo Principal de Eventos

Paso	Acción del Actor	Respuestas del Sistema
1	El Administrador ingresa al módulo de “Reportes” o al “Dashboard”.	El sistema consulta las tablas sales, tickets, products y financial_transactions para calcular totales y promedios.
2	Presenta los KPIs consolidados (Ingreso por reparaciones, ventas de accesorios, equipos en taller por estado y alertas de inventario).	Muestras gráficas y tablas dinámicas de datos.
3	El Administrador aplica filtros por rango de fechas (date_from, date_to) y selecciona el tipo de reporte deseado.	Filtra los registros en tiempo real en la tabla de vista previa.
4	Presiona el botón “Exportar PDF” o “Exportar a Excel”.	El servidor procesa la petición con los filtros aplicados.
5		Si es PDF: compila la plantilla corporativa mediante laravel-dompdf con encabezados institucionales, totalizadores y firma de auditoría.
6		Si es Excel: Genera un archivo binario .xlsx con encabezados, celdas tipadas para importación en hojas de cálculo y sumatorias automáticas.
7		Fuerza la descarga del documento en el navegador del Administrador.